



UniNDP: A Unified Compilation and Simulation Tool for Near DRAM Processing Architectures

Tongxin Xie^{*†}, Zhenhua Zhu^{*†‡}, Bing Li[§], Yukai He[◇], Cong Li[¶], Guangyu Sun[¶],
Huazhong Yang[†], Yuan Xie[‡], and Yu Wang[†]

[†]Dept. of EE, BNRist, Tsinghua University, [‡]HKUST, [§]Institute of Microelectronics, Chinese Academy of Sciences,

[◇]Capital Normal University, [¶]Peking University

^{*} Both authors contributed equally to this work.

Corresponding Authors: zhuzhenhua@mail.tsinghua.edu.cn, yu-wang@tsinghua.edu.cn

Abstract—Near DRAM Processing (NDP) architectures have emerged to be a promising solution for commercializing in-memory computing and addressing the “memory wall” problem, especially for the memory-intensive machine learning (ML) workloads. In NDP architectures, the Processing Units (PUs) are distributed next to different memory units to exploit the high internal bandwidth. Therefore, in order to fully utilize the bandwidth advantage of NDP architectures for ML applications, meticulous evaluations and optimizations of data placement in DRAM and workload scheduling among different PUs are required. However, existing simulation and compilation tools face two insuperable obstacles to achieving these targets. On the one hand, tools for traditional von Neumann architectures only focus on the data access behaviors between the host and DRAM and treat DRAM as a whole part, which cannot support NDP architectures with multiple independent processing and memory units working simultaneously. On the other hand, existing NDP simulators and compilers are designed for specific DRAM technology and NDP architecture, lacking compatibility for various NDP architectures.

In order to overcome these challenges and optimize data mapping and workload scheduling for different NDP architectures, we propose UniNDP, a unified NDP compilation and simulation tool for ML applications. Firstly, we propose a unified tree-based NDP hardware abstraction and the corresponding instruction set, enabling the support for various NDP architectures based on different DRAM technologies. Secondly, we design a cycle-accurate and instruction-driven NDP simulator to evaluate hardware performance by accurately tracking the working status of memory elements and PUs. The accurate simulation can provide effective guidance for compilation. Thirdly, we design an NDP compiler that optimizes data partition, mapping, and workload scheduling in different DRAM hierarchies. Furthermore, to enhance the compilation efficiency, we propose a hardware status-guided search space pruning strategy and a fast performance predictor using DRAM timing parameters. Extensive experimental results show that, compared to existing mapping and compilation methods, UniNDP can achieve 1.05-3.43 $\times$ speedup across multiple NDP architectures and different ML workloads. Furthermore, based on the results of UniNDP, we provide insights for the future NDP architecture design and deployment in ML applications.

I. INTRODUCTION

In recent years, machine learning (ML) algorithms, such as neural networks (NNs) [22], [66] and large language models (LLMs) [6], [47], [64], [65], have shown remarkable ability across various fields, such as computer vision [13] and natural language processing [8]. Despite the superior performance, the

amount of compute and model size of ML algorithms are increasing exponentially [4], [69], causing a large amount of data transfer between memory and processing units. What is worse, the improvement of hardware computational performance significantly outpaces the growth of DRAM bandwidth [19]. As a result, the heavy data access has become a critical bottleneck that limits the overall system performance and causes severe “Memory Wall” problem, which accounts for more than 60% latency in traditional von Neumann architectures [45] and underutilized computational capability [50].

Near DRAM processing (NDP, and also known as Near Data Processing) architectures have been proposed as a promising solution to this problem by placing processing units (PUs) inside main memory [2], [3], [7], [10], [11], [15], [17], [18], [23], [26], [28], [29], [32], [35], [36], [38], [39], [49], [54], [60]–[62], [71], [73], [77], and it has become a trend for industrial and commercial processing-in-memory (PIM) products [26], [28], [35], [36]. In NDP architectures, PUs can read/write data directly from nearby memory elements, and different PUs work independently, fully exploiting the high internal bandwidth of DRAM and ensuring that different memory units do not compete for the external bus. Furthermore, NDP architectures only transmit computation results via the bus to the host, thus reducing the data transfer volume between the host and main memory. Existing work has shown that the effective bandwidth of NDP architectures can be improved by one to two orders of magnitude, effectively relieving the memory wall problem [20], [35], [36].

Researchers have proposed various NDP architectures, which can be classified into four categories according to the placement position of PUs and the memory space visible to one PU, as shown in Fig. 1: (1) *Bank-level NDP*: each PU can only access memory elements within a single DRAM bank [7], [18], [26], [62], [77]. For example, the PU in UPMEM [26] can only access DRAM and buffer in its bank, and inter-bank data movements are required to access data residing in other banks. (2) *Device-level NDP*: the PU is placed near the bank and can access data from other banks within one DRAM device [15], [23], [35], [36], [38], [39], [49], [54], [71], [73]. For example, in Hyix GDDR-based Accelerator-in-Memory (AiM) [35], each PU can access data from other banks directly or through the device-level global buffer. (3)

Rank-level NDP: the PU is placed outside the device and communicates with memory units in one rank [11], [29], [32], [60]. For example, in Dual-Inline Memory Module (DIMM)-based NDP architectures, no internal circuit modifications to the DRAM device are needed and different ranks in one channel can work simultaneously using rank-level PUs and controllers [11]. (4) *Channel-level NDP*: the PU reads and writes data similarly to conventional main memory, while the major difference is that computation occurs before the data is transmitted through the external bus [2], [3], [10], [17], [18], [28], [49], [61], [73]. Besides these four categories, an NDP architecture can also contain PUs at multiple DRAM hierarchy levels [18], [49], [73].

Despite the diverse and complex architecture designs, all NDP architectures need meticulous optimizations in the following three aspects to leverage the high internal bandwidth:

- 1) **Data partition**: split data into several parts according to the PU number and memory capacity, achieving workload balance among PUs and efficient computing resource utilization.
- 2) **Data mapping**: determine *where* and *how* to store data. On the one hand, store data closely around their “consumer” PU to exploit the highest effective bandwidth. On the other hand, optimize the data layout inside the DRAM array to reduce the high-cost row buffer miss.
- 3) **Workload scheduling**: optimize input data sources, output results destination, and the pipeline of data access and computation for each operator, reducing the waiting time of PUs.

However, existing evaluation and compilation tools face two severe challenges to support these optimizations for various NDP architectures and ML workloads. On the one hand, compilers for traditional von Neumann architectures [9], [33], [34], [63] are unaware of the internal data access, communication, and computation inside NDP architectures. These tools treat DRAM as an entire pure memory and use DRAM simulators [30], [40]–[44] with computation-oriented simulators [5], [25], [48], [70], [74] to model the data communication overhead between host and main memory. As a result, they can not support NDP architectures with multiple independent PUs and memory units working concurrently.

On the other hand, many compilers [1], [7], [27], [55], [57] and simulators [16], [26], [37], [46], [51], [53], [58], [67], [68], [72], [75], [76] for NDP architectures have been proposed. Most of the compilers and simulators are designed for a specific NDP architecture and lack the compatibility with other NDP architectures. However, the same data partitioning and mapping strategy may show a significant performance gap on different NDP architectures considering different timing parameters, DRAM organizations, and PU placements. Therefore, a unified simulator and compiler for various NDP architectures is required, which can generate the optimal data partitioning, mapping, and workload scheduling strategies corresponding to different NDP architectures and provide guidance for the future NDP architecture design.

In this paper, we propose UniNDP, a unified NDP compilation-simulation tool for ML workloads. The contributions of this work are as follows:

- We propose a unified tree-based hardware abstraction for NDP architectures, which can be easily configured to various NDP architectures. Based on the unified NDP abstraction, we summarize and design a unified NDP instruction set to cover all possible compute and data movement commands. The abstraction and instruction set ensure the support for various NDP architectures based on different DRAM technologies.
- We design a cycle-accurate and instruction-driven NDP simulator to provide accurate evaluation by tracking the working status of PUs, memory units, buffer, and internal buses. It can also support instruction re-order and out-of-order execution within NDP to assess the attainable upper bound of the hardware performance by improving the computational parallelism and bandwidth utilization.
- We propose an NDP compiler with a DRAM-hierarchy-aware encoding method to optimize data partition, mapping, and workload scheduling in different DRAM hierarchies. We further propose a hardware status-guided search space pruning strategy to reduce the compilation search space. A fast NDP performance predictor using DRAM timing parameters is also designed to reduce the evaluation time from 90s to 66 μ s during compilation.

Extensive experiments on a series of ML models have shown that, compared to existing mapping and compilation methods, UniNDP can achieve 1.86-3.43 $\times$ and 1.05-1.49 $\times$ end-to-end speedup for CNNs and LLMs on different NDP architectures, respectively. By leveraging UniNDP, we further provide several insights for the future NDP architecture design and ML workload deployment.

II. PRELIMINARY AND MOTIVATION

A. Demand for Unified NDP Compilation Tool

NDP architectures provide promising solutions to overcome the memory wall problem for data-intensive applications. To fully leverage the advantages of NDP architectures, an NDP-specific compilation tool is in high demand that optimizes data partition, data mapping, and workload scheduling. However, various hardware parameters affect the optimized compilation results for NDP architectures.

At the device level, different DRAM technologies, such as Double Data Rate Synchronous DRAM (DDR SDRAM), Graphics DDR (DDR), High Bandwidth Memory (HBM), etc., have different timing parameters and organizations. As a result, the accessed data bitwidth and data read/write latency/bandwidth of each PU vary a lot in different DRAM technologies. Furthermore, the number of accommodated PUs is also influenced by the DRAM organization (i.e., internal memory hierarchy). For example, in DIMM-based rank-level NDP architectures [12], [29], the PU fetches 64-bit data at once from DRAM devices in one rank with the bandwidth of ~ 20 GB/s. While in HBM-PIM [36], one bank-level PU

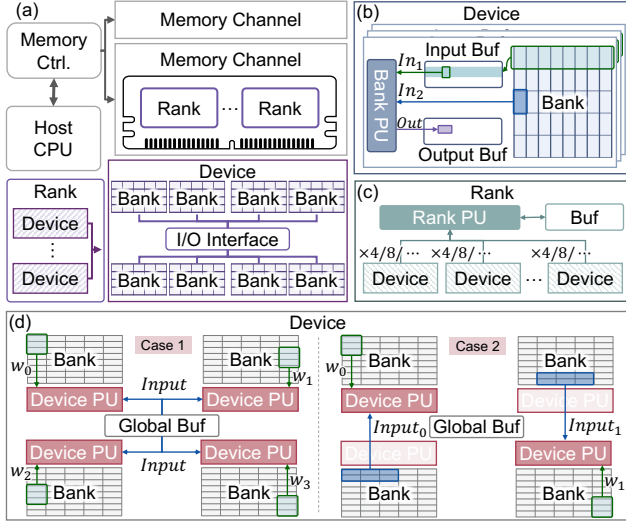


Fig. 1. (a) Basic DRAM organization structure; (b) Bank-level NDP architecture; (c) Rank-level NDP architecture; (d) Device-level NDP architecture.

accesses 256-bit data and a total of 4096-bit data is accessed across all the banks in parallel, achieving $\sim 1\text{TB/s}$ effective bandwidth. The different data access granularities, bandwidths, and computational parallelisms affect the optimal data partition size and pipeline parallelism strategies for various NDP architectures.

At the PU level, the position of PU determines the visible memory space and the total PU number, further influencing the optimal data partition and mapping strategy. To be specific, for bank-level and device-level PUs, the visible address space is mainly confined to a single DRAM bank or device, limiting the directly accessible data capacity (e.g., 512MB). For example, UPMEM [20] accesses data from other banks through the host. Besides, bank-level NDP architectures contain a large number of PUs, enabling high computational parallelism among PUs. On the contrary, channel-level and rank-level PUs have a larger visible memory space (e.g., 64GB), making them suitable for applications with larger data storage requirements, such as recommendation systems [29], [32]. Regarding to the computing power, high-level NDP architectures have more relaxed constraints on hardware area and power of PUs, leading to high computational parallelism inside each PU.

At the data path level, some NDP architectures place local/global buffers within the main memory, adjusting the location of inputs and outputs in the data path to reduce direct access to DRAM. Different data paths require different workload scheduling strategies to improve buffer utilization and hit rate. For example, the PUs in UPMEM are equipped with local input & output buffer [20], i.e., it can compute using operands from input buffer and bank, with the results stored in the output buffer. While in AiM [35], each bank-level PU does not have a local buffer, but the device is equipped with a global buffer that can broadcast to all banks through device bus. Different bank-level PUs can perform computations using

TABLE I
ABBREVIATIONS OF DRAM TIMING PARAMETERS USED IN THIS PAPER.

Abbr.	Meaning	Abbr.	Meaning
t_{CCD}	Column-to-column delay	t_{RCD}	Row-to-column delay
t_{RP}	Row precharge time	t_{WR}	Write recovery time
t_{RTP}	Read to precharge time	t_{WTR}	Write to read time
RL	Read latency	WL	Write latency
t_{REF}	Refresh interval	t_{RTRS}	Rank-to-rank switch time
t_{RAS}	Row access strobe	t_{RRD}	Row-to-row activation delay
t_{CK}	Clock cycle time	BL	Burst length

the same operand from the global buffer. For these two NDP architectures, UPMEM needs to optimize the buffer usage of each bank independently, while AiM needs to consider data reuse and partitioning of the global buffer across multiple banks during workload scheduling.

Based on the above analysis, NDP architectures with different DRAM technologies, PU designs, and data paths require different compilation optimization strategies, bringing an urgent need to a unified compiler for various NDP architectures.

B. Basis of DRAM Performance Modeling

During the NDP architecture compilation, we need to evaluate the hardware performance to guide the compilation optimization. In NDP architectures, since computation and DRAM access are tightly combined, the key point of performance evaluation is to model the data access behavior of DRAM and incorporate it with latency of computation and communication.

During data access of DRAM, several types of DRAM commands are issued by a memory controller (MC), including: (1) Activation (ACT) commands to activate a DRAM row and place the data into row buffer before it can be accessed; (2) READ and WRITE commands to access a group of columns in an open row (i.e., data stored in row buffer); (3) Precharge (PRE) commands to close a row and charge the bitline before accessing data in another row (i.e., row buffer miss).

DRAM commands take dozens of cycles to finish, and should obey strict timing constraints. Table. I lists parameters of DRAM timing constraints considered in this work. DRAM memory controller use these parameters to determine when to issue the next command. For example, READ/WRITE command should be issued t_{RCD} cycles after the previous ACT command.

III. OVERVIEW OF UNINDP

In order to optimize data partition, data mapping, and workload scheduling for different NDP architectures, we propose UniNDP, a unified compilation and simulation tool for NDP architectures. Fig. 2 shows the overview of UniNDP. UniNDP mainly focuses on the machine learning workloads, especially for NNs and LLMs.

The input of UniNDP contains two parts: workload representation and hardware configuration. The workload is represented by a series of operator-level intermediate representations (IR), which is the output of ONNX framework. Among all operators of NNs and LLMs, we choose to deploy the most time-consuming and memory-intensive operators in NDP

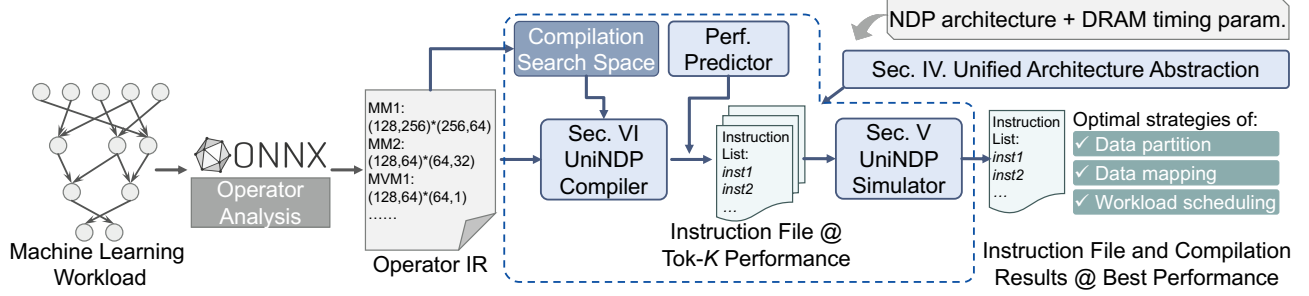


Fig. 2. Overview of UniNDP.

architectures, such as Matrix-Vector Multiplications (MVM) and Matrix-Matrix Multiplications (MM). The hardware configuration includes detailed DRAM timing parameters that represent different DRAM technologies and the architecture specifications. In UniNDP, we propose a unified tree-based hardware abstraction that can be used to describe different NDP architectures, which will be described in Section IV.

After inputting the workload IR and hardware configuration, UniNDP will first create the partition, mapping, and scheduling search space for the workload. Then it will explore the search space with a working status-guided space pruning strategy, reducing the search space size. Here the “working status” contains the number of rows and columns read from the DRAM, as well as the number of DRAM row precharge. To further improve the search efficiency, UniNDP uses a fast performance predictor based on DRAM timing parameters to find the optimal compilation results with top- K predicted performance. After that, UniNDP will generate an instruction sequence for these top- K compilation strategies, based on the instruction set we defined in Section IV-B. The details of UniNDP’s compiler will be described in Section VI.

Then the cycle-accurate simulator of UniNDP is called iteratively to evaluate the accurate performance of each top- K instruction sequence. The proposed simulator contains a virtual memory controller to support instruction re-order and out-of-order instruction issue, which exploits and evaluates the attainable upper bound of these instruction sequences. According to the simulation results, the strategies of data partition, mapping, and workload scheduling with the best performance will be reported. Furthermore, UniNDP’s simulator also serves as the reference to adjust and validate the proposed fast performance predictor. The details of UniNDP’s simulator will be discussed in Section V.

IV. UNIFIED NDP ABSTRACTION AND INSTRUCTION SET

A. Hardware Abstraction for NDP Architectures

To make the compilation and simulation tools universally applicable to various NDP architectures, we first design a unified hardware abstraction that can describe different NDP architectures, as shown in Fig. 3.

We adopt a scalable tree structure to abstract the NDP architecture, which can be used to describe existing NDP

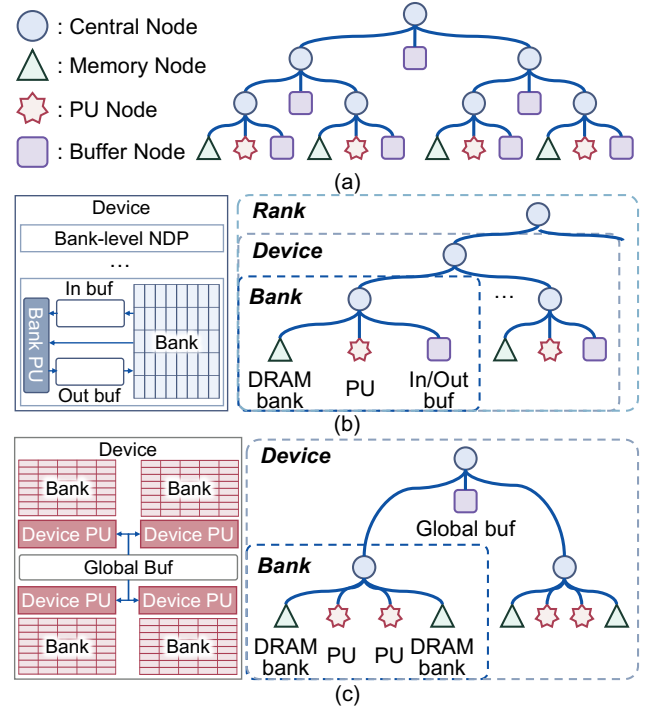


Fig. 3. (a) Tree-based NDP abstraction; Example of (b) bank-level and (c) device-level NDP architectures.

architectures and extended to support new NDP designs. Each level of the tree represents a hierarchical layer of DRAM, e.g., channel, rank, device, and bank. New DRAM hierarchy like bank group can be extended by introducing tree nodes in a new level, and hierarchy that does not exist in NDP architecture can also be removed, such as the device level in HBM-PIM [36].

Each level of the NDP abstraction tree contains multiple virtual **central nodes**. These central nodes represent the interface for transmitting data from the lower level to the higher level (e.g., data buses). The central nodes have three kinds of children nodes: the central nodes of its lower level (for non-bottom levels) or memory nodes (for the bottom level), PU nodes, and buffer nodes. The **memory node** is the smallest memory unit containing a PU in an NDP architecture; for

example, in bank-level NDP, each memory node represents a bank. The memory node contains the attributes of read/write bitwidth, capacity, and timing parameters. The **PU node** and **buffer node** stand for the PU and input/output buffer in the corresponding DRAM hierarchical layer, which are both optional in the tree structure.

The PU node includes the key parameters of computation parallelism and operating frequency. In this paper, we primarily focus on the matrix-related operators. PU nodes can also be extended to support the description of other functions, such as vector summation [29] and set operations [12]. Besides, if a central node contains multiple memory nodes and PU nodes, each PU node can read/write data from/to multiple brother memory nodes, and the specific memory nodes it can access can be configured by users.

The buffer node contains the attributes of buffer size (width and depth) and read/write latency. The buffer node can be configured as the local buffer of its brother PU node and the global buffer shared by multiple PUs at a lower tree-level. Therefore, two brother PU nodes can share the same input from their brother buffer node (i.e., local buffer), and two PU nodes from different central nodes should use the upper-level memory node (i.e., global buffer) for data communications.

By adjusting the tree configuration, the number of central nodes, memory nodes, PU nodes, and memory nodes in each tree-level, we are able to cover various kinds of NDP architectures. Fig. 3(b) and (c) show the representation examples of bank-level NDP architectures [20] and device-level NDP architectures [35].

B. Unified NDP Instruction Set

Based on the tree-based NDP abstraction, we further propose the unified NDP instruction set to cover all possible operations used in NDP architectures, as shown in Table. II.

The unified NDP instruction set consists of three types of commands: compute instructions, inside NDP data movement instructions, and host-side operation instructions. It should be noted that the instructions listed in Table. II will be further split into a series of DRAM commands (e.g., ACT, READ, WRITE, etc.) by the memory controller during runtime.

For the compute instructions, a PU can compute using input operand *A* from DRAM (e.g., one bank) and operand *B* from DRAM (e.g., another bank), global buffer, and PU's local input buffer. The MAC instruction is a vectorized instruction, which access multiple data from continuous DRAM columns in one DRAM row using a single instruction, reducing the overhead to command and address (C/A) bus. Besides, each MAC instruction requires a certain hardware support. For example, bank-level MAC-DRAM needs the data path connecting two different banks to a PU; MAC-LB requires the PU should be equipped with a local buffer. Therefore, if certain hardware components are missing in the tree-based NDP abstraction, the UniNDP compiler will automatically exclude the corresponding instructions during the compilation process.

The data movement instructions exchange data inside NDP architectures, including three kinds of instructions: (1)

copy data between PU registers and local buffer (REG2LB, LB2REG), which is typically used for storing and updating temporary results; (2) copy data between local buffer and DRAM (LB2DRAM, DRAM2LB) to leverage data locality to reduce DRAM access; (3) transfer data between global buffer and DRAM (GB2DRAM, DRAM2GB) to enable PU to fetch data from addresses that are not visible to it.

The host-side operation instructions control the data communication between CPU and NDP architectures. Besides the conventional DRAM read/write instructions (i.e., READ-DRAM, WRITE-DRAM), UniNDP adds another three kinds of instructions to achieve high parallel communication between the host and all PUs within the NDP architecture. In UniNDP, we assume that the host can perform parallel write operations to the PU's registers (WRITE-REG), local buffer (WRITE-LB), and global buffer (WRITE-GB) to reduce internal data transmission within NDP architectures and improve the efficiency of each PU in fetching input data. While for data readout, considering that the NDP architecture only needs to return the final results to the host, we only reserve the instructions for data transfer from the result registers to the host (READ-REG).

With this instruction set, we are able to cover different dataflows of various NDP architectures. Here we take UPMEM and AiM as examples to explain how the NDP instructions change in different architectures. For UPMEM, a WRITE-LB instruction is used to prepare the input matrix, while in AiM, this process is alternated by a WRITE-GB or WRITE-DRAM instruction since AiM does not contain PU-level local buffer. When the input data are ready, either MAC-LB, MAC-GB, and MAC-DRAM instruction are used for computation. It should be noticed that in AiM, MAC-DRAM will reduce the activated PU number to half of the DRAM bank number (i.e., two DRAM banks activate one PU), so it will lose half of the parallelism compared to MAC-GB. When getting some intermediate results, the REG2LB instruction is used to store the intermediate result from PU register to the output buffer. After obtaining the complete results, LB2DRAM is used to write the partial result back to DRAM for further process. While in AiM that does not have output buffer, LB2DRAM is alternated by READ-REG and WRITE-REG to return the results to the host.

V. CYCLE-ACCURATE & INSTRUCTION-DRIVEN NDP SIMULATOR

A. Overview

The simulator is responsible for evaluating the instruction sequence generated by the compiler. It is composed of three parts: (1) a hardware status space to simulate and track the working state of DRAM and other hardware components like buses, PUs, and buffers; (2) an instruction queue to accommodate instruction sequence provided by the compiler and maintain the data dependency; and (3) a virtual memory controller (VMC) module to decide which instruction should be issued first in the instruction queue, and when to issue this instruction, based on the hardware status space.

TABLE II
UNIFIED INSTRUCTION SET FOR NDP

Type	Instruction	Description
Compute	MAC-DRAM	Perform MAC operations, both input operands from DRAM, store results in register
	MAC-GB	Perform MAC operations, inputs from DRAM and global buffer, store results in register
	MAC-LB	Perform MAC operations, inputs from DRAM and local buffer of PU, store results in register
Data Movements	REG2LB, LB2REG	Copy data between result/input register and local buffer of PU
	LB2DRAM, DRAM2LB	Copy data between local buffer of PU and DRAM
	GB2DRAM, DRAM2GB	Copy data between global buffer and DRAM
Host-side Operations	READ-DRAM, WRITE-DRAM	Read/write data from/to DRAM as conventional memory
	WRITE-GB	Write data to the global buffer from the host
	WRITE-LB	Write data to the local buffer of PU from the host
	READ-REG, WRITE-REG	Read/write MAC results/input from/to PU register to/from the host

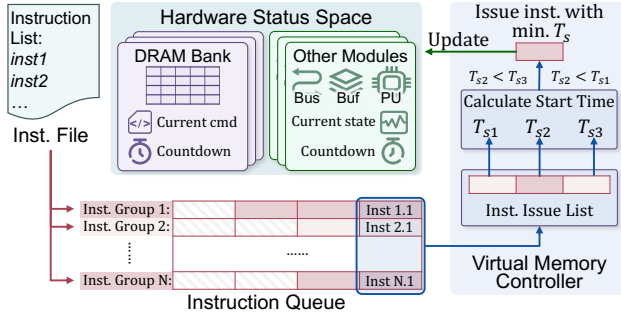


Fig. 4. Overview of UniNDP simulator.

The proposed NDP simulator works in an instruction-driven manner, which means that the hardware status space is not updated cycle-by-cycle, but updated only when a new instruction is issued. Therefore, the number of simulation iterations will be reduced from the number of DRAM cycles to compute the workload to the length of the instruction sequence, bringing $10 \sim 300 \times$ less simulation time on different NDP architectures. Each simulation iteration contains four steps, as shown in Fig. 4:

Step 1: The instruction queue will first check the data dependency of instructions and send issuable instructions $\{INST_i\}$ to the VMC module.

Step 2: Then VMC will check the possible issue time for each instruction $INST_i$ based on the hardware status space, and choose the instruction with the earliest issue time T_{issue} as the next instruction to issue.

Step 3: Because no other instructions can be issued before T_{issue} , the hardware status space is directly updated to the time of T_{issue} .

Step 4: Finally, the MC will issue this instruction by deleting it from the instruction queue, and update the status of occupied hardware based on its hardware performance model.

These four steps will be repeated until all instructions from the instruction queue is issued and the hardware status space is free, and this time will be returned as the latency of the entire workload. Furthermore, the NDP simulator can be easily extended to support non-ML workloads by modifying PU performance models (e.g., use Gem5 [5] for near-memory

CPU or other general-purpose computing units), adding the corresponding IR, and reusing the memory access modeling.

B. Hardware Status Space

To provide cycle-accurate simulation for NDP architectures, the DRAM timing constraints should be precisely modeled and considered during simulation. Therefore, we build up a hardware status space to track the working states of different components. The update of hardware status space is triggered by DRAM commands according to the timing parameters.

Firstly, we track the activation states of each DRAM bank. Considering the DRAM READ/WRITE commands are not allowed when a bank is not activated, an ACT command should be scheduled before accessing an inactivated memory. Secondly, the address of the currently activated DRAM row is also tracked to indicate if the following memory access triggers a row buffer miss (i.e., row address altered). In this occasion, a PRE and a following ACT should be scheduled to activate another DRAM row. Thirdly, other timing constraints between different DRAM commands are also considered by maintaining four countdowns to tell when the next ACT, PRE, READ, and WRITE command can be scheduled. Last but not least, we also use countdowns to track the idle/occupied states of PUs, buses, and buffers. All countdowns are updated with the global tick. When a new instruction is issued, the countdowns for the involved hardware resources are reset based on the specific DRAM command or the resource occupation time.

C. Instruction Queue

Data dependency is the most noteworthy factor for the instruction-driven simulation. The NDP simulator must follow two basic rules for issuing and evaluating instructions: (1) the operand data should be prepared before an instruction is issued, (2) the memory space of the data should not be reused until this instruction is finished.

Existing memory-oriented DRAM simulators adopt memory barriers for DRAM Read/WRITE commands scheduling to obey data dependency. However, memory barrier designs are inefficient for NDP architectures for two reasons. On the one hand, in conventional memory, different DRAM devices in one rank or different memory banks in one device work in lockstep. But in NDP architectures, different PUs work independently, resulting in a significant increase in the number

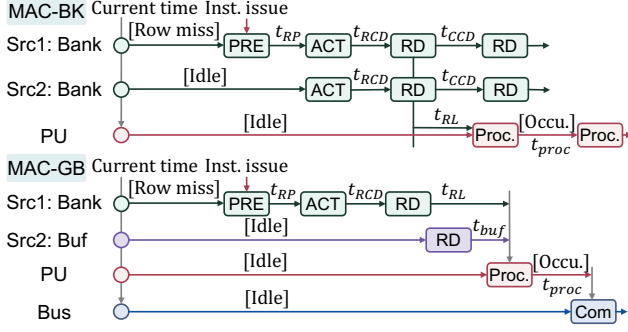


Fig. 5. Examples of instruction execution.

of instructions. As a result, using memory barriers in this case will cause heavy burdens to the size of code files. On the other hand, in NDP architectures, the data dependency may only occur within a local scope of certain instructions. For example, in bank-level NDP, data dependency between input update and computation only exists between instructions of the same bank. However, using memory barriers to enforce synchronization across different banks will lead to bubbles and under-utilization of PUs.

To overcome these challenges, we design an instruction queue to schedule the order of instructions, ensuring the instruction issue and the corresponding simulation obey the data dependency. The instruction queue is composed of different instruction groups. Each instruction group contains multiple instructions with data dependencies, like instructions of “input-compute-output”. While the instructions of individual and independent hardware are placed into different groups, e.g., instructions for different banks in bank-level NDP. Therefore, different instruction groups can be issued in parallel. In case that independent PUs and other hardware components need data communication and synchronizations, the data dependencies between two instruction groups can also be configured. During each iteration, the instruction queue will first select the groups without predecessors in the data dependency graph, and collect the first instruction in each group to the VMC.

D. Virtual Memory Controller (VMC)

In order to fully exploit and assess the attainable upper bound of NDP performance, the proposed NDP simulator contains a VMC to support instruction re-order and out-of-order execution. The VMC is responsible for scheduling DRAM commands and orchestrating other hardware resources like PUs, buffers, and buses. It mainly accomplishes two functions: (1) find the earliest time when a given instruction can be issued, and (2) set the resource countdowns correctly to reveal the influence of this instruction if it is issued.

Here we take MAC-DRAM and MAC-GB as examples to show how the VMC and simulation flow work, as shown in Fig. 5. To get the issue time T_i of each instruction, we first determine the earliest time that PU can start computation (T_S), by both checking the data arrival time T_A from different

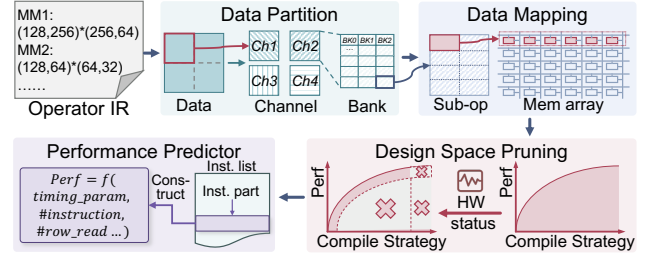


Fig. 6. Overview of UniNDP compiler.

sources, and the free time of PU itself ($T_F(PU)$). For operands from **DRAM bank** BK_i , the row activation state is read to check if ACT or PRE command should be inserted before the memory access, then the data arrival time $T_A(BK_i)$ can be calculated by adding DRAM countdowns, read latency, and the timing intervals of the inserted PRE and ACT commands, which is shown in Fig. 5(a). For operands from the **global buffer**, the data arrival time $T_A(GB)$ is determined by the released time and read latency of the global buffer, as well as when the bus is idle for transmission, as shown in Fig. 5(b).

$$T_A(GB) = \max(T_F(GB) + RL(GB), T_F(BUS)). \quad (1)$$

Then the start time of the computation can be determined:

$$T_S = \max\{T_A(GB), T_A(BK_{0,1,\dots}), T_F(PU)\}. \quad (2)$$

Finally, DRAM commands are scheduled as-late-as-possible regarding the deadline of T_S . The issue time of the earliest DRAM command (PRE, ACT, READ, etc.) will be considered as the issue time for this instruction. Among all issuable instructions, the one with the earliest issue time will be chosen to issue. The DRAM countdowns and state are updated according to the T_{Start} , auto-precharge policy, and the number of continuously fetched columns of the chosen instruction. And the countdown of other hardware components will also be set based on the occupied duration.

VI. UNIFIED NDP COMPILER

A. Overview

The compiler is designed to find optimal strategies for deploying a workload on the given NDP architecture, which is achieved by three key steps: strategy encoding, search space pruning, and predictor-based search, as demonstrated in Fig. 6. Firstly, we design encoding methods for partition and mapping strategy, which can describe: (1) how the computations and data of the workload are partitioned to PUs and DRAMs across multiple DRAM hierarchies, (2) how the operand matrix is sliced and mapped to different DRAM column and rows, i.e., the data layout. Leveraging these encoding methods, we can delineate the search space of the compilation strategy. Secondly, we reduce the search space size with a hardware-status-guided space pruning method, which is designed based on the correlations between the end-to-end latency and hardware states-related indicators like the PU utilization and the number

of DRAM commands (e.g., READ/WRITE to access data and PRE/ACT to change the row). Then, a fast performance predictor based on DRAM timing parameters are utilized to find the optimal compilation results with top- K predicted performance, and generate instruction sequence for these top- K compilation strategies. Finally, we use the cycle-accurate simulator to accurately simulate the performance of each top- K instruction sequence and report the best compilation strategy.

B. Encoding Method for Partition

In order to achieve high parallelism, existing NDP architectures use the data partition strategy along the matrix dimensions that do not need reduction and the partition number equals to the number of available PUs [31], [35], [36], [51]. Here we take a matrix multiplication operator $O = W \times A$ as the example, the dimensions of the weight matrix W , the input activation matrix A , and the output matrix O are (M, K) , (K, N) , and (M, N) , respectively. The existing strategy splits data and workload along the M dimension. However, when the dimensions of K and N grow up, the number of updates to the input buffer also increases, hindering the high computing parallelism because of more input data access.

Facing this issue, we unveil the hidden optimization opportunities in partition strategies through a new partition and encoding method, which allows the data to be partitioned along more than one dimensions and processed across multiple DRAM hierarchies. The proposed partition encoding method sequentially describes how the operator and data are split at various DRAM hierarchy levels, and how they are partitioned in different dimensions at each DRAM hierarchy.

When we partition the MM operator on a bank-level NDP architecture with $\{\#CH, \#RA, \#DE, \#BA\}$ parallelism across different hierarchies, $\#CH, \#RA, \#DE, \#BA$ also mean the number of channels, ranks per channel, devices per rank, and banks per device, respectively. The partition strategy can be represented as:

$$\begin{aligned} Part = \{ & Part_{CH}(M_{CH}, K_{CH}, N_{CH}), \\ & Part_{RA}(M_{RA}, K_{RA}, N_{RA}), \\ & Part_{DE}(M_{DE}, K_{DE}, N_{DE}), \\ & Part_{BK}(M_{BK}, K_{BK}, N_{BK}) \}. \end{aligned} \quad (3)$$

$Part_{\{CH, RA, DE, BK\}}$ represents four levels of partition corresponding to four DRAM hierarchies, and the partition will start from the highest channel level. To be specific, $Part_{CH}$ means that the operator is equally partitioned into M_{CH} , K_{CH} , and N_{CH} parts along the M , K , and N dimensions. Each partitioned operator is distributed to a different channel, and $Part_{CH}$ should satisfy the restriction that $M_{CH} \times K_{CH} \times N_{CH} \leq \#CH$ to avoid allocating more channels than available. Similarly, from channel to bank, we can partition an operator along multiple dimensions and to multiple DRAM hierarchies. Then, we can use the 12-tuple to represent the partition results: $\{\{M, N, K\}_{\{CH, RA, DE, BK\}}\}$

We denote the operator after all partitions as a sub-operator, whose size can be represented as $(M_{Orinal}, K_{Orinal}, N_{Orinal})$ are the original matrix sizes):

$$\begin{aligned} M_{Sub} &= \left\lceil \frac{M_{Orinal}}{M_{CH} \times M_{RA} \times M_{DE} \times M_{BK}} \right\rceil, \\ K_{Sub} &= \left\lceil \frac{K_{Orinal}}{K_{CH} \times K_{RA} \times K_{DE} \times K_{BK}} \right\rceil, \\ N_{Sub} &= \left\lceil \frac{N_{Orinal}}{N_{CH} \times N_{RA} \times N_{DE} \times N_{BK}} \right\rceil. \end{aligned} \quad (4)$$

C. Encoding Method for Mapping

After partitioning the data of the operator, the next step is to encode how the sub-operators of weight, input, and output are mapped to the DRAM rows and columns in each bank. In modern DRAMs, multiple columns can be accessed continuously using a single READ/WRITE command, the number of which is called burst length (BL). We treat the multiple columns of data corresponding to the burst length as the smallest data block read from DRAM, which is used for data mapping. The mapping strategy can be represented as:

$$\begin{aligned} Map &= \{ Map_W((M_W^B, K_W^B), (M_W^R, K_W^R)), \\ & Map_A((K_A^B, N_A^B), (K_A^R, N_A^R)), \\ & Map_O((M_O^B, N_O^B), (M_O^R, N_O^R)) \}. \end{aligned} \quad (5)$$

$Map_{\{W, A, O\}}$ represents the mapping strategy for weight, input activation, and output. Take Map_W as an example, the weight matrix data of the sub-operator is firstly split into multiple data slices, each of which will be mapped to one data block. The size of each data slice is $M_W^B \times K_W^B$, thus the restriction is:

$$M_W^B \times K_W^B \times DataBits \leq BL * DeviceWidth, \quad (6)$$

where $DataBits$ and $DeviceWidth$ are the bitwidth of each matrix data and the bitwidth of one DRAM column, respectively. After that, multiple data slices with the number of $M_W^R \times K_W^R$, M_W^R slices along the M dimension and K_W^R slices along the K dimension, are mapped to the same row. Then we have another restriction:

$$BL \times M_W^R \times K_W^R \leq \#Col \text{ per Row}, \quad (7)$$

Similar to data partition, we use 12-tuple to encode the data mapping results: $\{M_{\{W, O\}}^B, K_{\{W, A\}}^B, N_{\{A, O\}}^B, M_{\{W, O\}}^R, K_{\{W, A\}}^R, N_{\{A, O\}}^R\}$.

D. Hardware Status-Guided Search Space Pruning Method

Considering the choices of data partition, mapping, and workload scheduling, the search space size of compilation strategies is tremendous. To tackle this problem, we propose the hardware status-guided search space pruning method, which contains two steps. In the first step, we constrain the range of mapping encodings based on the two hints:

- We need to fully utilize the computational parallelism of PU reduction units (e.g., adder tree) to enhance computational efficiency. Thus, the K dimension size of one data slice (i.e., K_W^B & K_A^B) should be set as close as possible to the computational parallelism of the reduction unit.

- K_W^R and K_A^R are suggested to be the same to match the dimensions of data read from DRAM, thereby reducing the number of redundant reads.

In the second step, we analyze the NDP performance upper bound using the hardware status, which is used as an indicator to dynamically reduce the search space during runtime. The performance upper bound is shown below:

$$Perf_{max} = \frac{\#DRAM\ Access}{Bandwidth} + \#Row_{miss} \times t_{RP} + \frac{Workload}{\#PU \times Perf_{PU}}, \quad (8)$$

where the first item calculates the minimum DRAM access time for PUs, the second term uses the amount of row buff miss to calculate the total precharge time, and the third term estimates the ideal latency of PU computation. The performance upper bound does not take extra data movements into consideration, e.g., data transfer among PU, buffer, and other PUs. That is to say, if the performance upper bound is slower than the performance threshold, then the compilation strategy can be skipped for further evaluation.

E. NDP Performance Predictor

The predictor is included in the compiler for two reasons. On the one hand, although the space of compilation strategies has been pruned, we still cannot afford the overhead to precisely evaluate the performance of each compilation strategy by timing-consuming cycle-accurate simulation. On the other hand, the theoretical analysis of ideal NDP performance upper bound is fast enough for dynamically design space pruning but not precise enough due to the lack of evaluation of extra data movement instructions. As a result, a fast and precise performance predictor for estimating the extra data movement latency is necessary, i.e., latency caused by data movement and host-side operation instructions in Table. II. However, the extra latency is hard to be theoretically analyzed without the instruction file, as it is influenced by various factors such as buffer size, data flow, and possible instruction overlaps in out-of-order execution. Therefore, we embed the predictor inside the code generation process, and the code generator utilizes a short and fixed-size part of the instruction sequence to capture the patterns of data movement instructions. After that, the predictor uses these patterns to predict the latency of this instruction part:

$$\begin{aligned} Lat_{partial} = & t_{CCD} * (\#READ + \#WRITE) \\ & + \#Row_{miss}(READ) * t_{RowChange}(READ) \\ & + \#Row_{miss}(WRITE) * t_{RowChange}(WRITE) \\ & + Lat_{Buffer/Reg}(Host), \end{aligned} \quad (9)$$

where the $\#READ(WRITE)$ represents the number of total DRAM READ/WRITE from the computing, data movement, and host instructions. $\#Row_{miss}$ and $t_{RowChange}$ represents the number and extra latency of row miss during continuous DRAM READ/WRITE, and the $Lat_{Buffer/Reg}$ takes into account the influence of host instructions on the data bus. Then the performance of the full instruction sequence can be derived according to the loop times of this partial instruction pattern.

$$TotalLat = \#Loops \times Lat_{partial} \quad (10)$$

VII. EXPERIMENTAL RESULTS

A. Experiment Setup

1) *Hardware Configuration*: The evaluation is performed on bank-level, device-level, and rank-level NDP architectures to evaluate the compatibility of UniNDP. The configurations of these architectures are listed in Table. III. For the bank-level NDP architecture (**Arch 1**), we follow the 20-channel design of UPMEM-PIM [26], but simplify the RISC-V core to a MAC unit for NN workloads. For the device-level NDP architecture, we both refer to the 64-channel design of Hynix AiM [31] and the 64-channel design of HBM-PIM (**Arch 4**) [36]. For AiM, we evaluate both the 16 PU mode (**Arch 2**) and 8 PU mode (**Arch 3**). For the rank-level NDP architecture (**Arch 5**), we reuse the memory system of UPMEM-PIM and use the architecture assumptions of DIMMining [11], where multiple rank-level PUs can compute simultaneously. All the PUs in these five architectures are configured to MAC units with the same computation parallelism and operating frequency to guarantee fair comparisons. The DRAM timing parameters we used refer to the Samsung PIMSimulator [51], [59], DRAMSim3 [43], and Ramulator2 [44].

2) *Workloads*: The workload are operators selected from both CNNs [13], [22], [56] and LLMs [65]. The supported operators include matrix operators (e.g., MM and MVM), vector operators (e.g., softmax and normalization), and element-wise operators (e.g., non-linear function and element-wise matrix-addition). For the CNNs, we converted the weight of the convolutional layer by img2col to evaluate the calculation of an output pixel by performing MVM on **Arch 1-5**. We also consider different stride, padding, and batch size settings for CNNs. For the LLMs, we select MM and MVM operators with different shapes from llama2_7B, llama2_13B, and llama2_34B [65], and test them with the max input length (i.e., 4096) on **Arch 1-5**. The information of these MM and MVM operators is listed in Table. IV. The computational power of the MAC unit is set to match the internal bandwidth of the corresponding DRAM hierarchy.

B. Performance Comparisons

To validate the effectiveness of UniNDP, we take the partition and mapping strategies of AiM and HBM-PIM [31], [35], [36], [51] as our baseline, which prioritizes the partition along the M -dimension, and requires that the partition size of the K -dimension which is mapped to a DRAM row should be as large as possible. We compare the compilation results of UniNDP and the baseline strategy of MM and MVM workloads, as shown in Table. V and Table. VI, respectively. From the results, we can show that UniNDP achieves $1.07\text{-}1.49\times$ and $1.10\text{-}1.62\times$ average speedup for MM and MVM workloads.

Fig. 7 shows the end-to-end performance improvement achieved by UniNDP for different CNNs and LLMs. Compared with the existing NDP partition and mapping strategies, UniNDP can achieve $1.86\text{-}3.43\times$ and $1.05\text{-}1.49\times$ speedup for CNNs and LLMs, respectively. Fig. 8 presents the key metrics of UniNDP's compilation results, normalized to the baseline.

TABLE III
NDP HARDWARE CONFIGURATIONS

	Arch 1	Arch 2	Arch 3	Arch 4	Arch 5
NDP level	bank	device	device	device	rank
# Channels	20	64	64	64	20
# Ranks/channel	2	1	1	1	2
# Devices/rank	8	1	1	1	8
# Banks/device	8	16	16	16	8
# PUs	1 (per bank)	16 (per device)	8 (per device)	8 (per device)	4 (per rank)
Buffer design	64 KB (local)	2KB (device-level global)	2KB (device-level global)	512B (local)	64 KB (local)
Memory technology	DDR4	GDDR6	GDDR6	Samsung-HBM2	DDR4
Timing parameter	tCK=0.83, tRCD=17, tRP=17, tCCD=6, BL=8, tCL=17	tCK=0.57, tRCD=26, tRP=26, tCCD=6, BL=16, tCL=24	tCK=0.57, tRCD=26, tRP=26, tCCD=6, BL=16, tCL=24	tCK=1, tRCD=14, tRP=14, tCCD=4, BL=4, tCL=20	tCK=0.83, tRCD=17, tRP=17, tCCD=6, BL=8, tCL=17
Rows, Columns	32768, 1024	16384, 1024	16384, 1024	32768, 1024	32768, 1024
Device width	x8	x16	x16	x64	x8

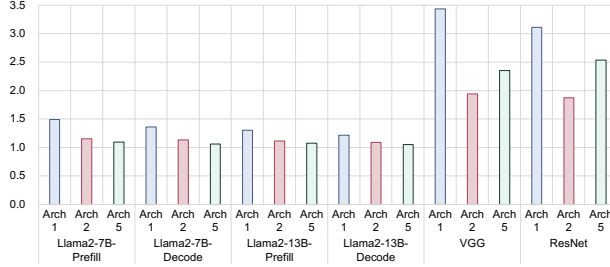


Fig. 7. End-to-end performance improvement (normalized to baseline).

UniNDP can reduce the number of DRAM access-related commands and DRAM row change (i.e., row activation) by up to 86%, which is the main reason why UniNDP outperforms existing compilation strategies. Compared to LLMs, the convolution computations using sliding window in CNNs have a higher ratio of the input matrix size to the weight matrix size. Existing NDP compilation strategies issue a large number of host-side instructions to schedule and handle large-scale input matrices, which offers UniNDP substantial optimization opportunities. Consequently, the performance improvement of UniNDP for CNNs is better than that for LLMs. Among different NDP architectures, UniNDP achieves the best speedup on **Arch 1**. The main reason is that **Arch 1**, as a bank-level NDP architecture, provides a larger optimization search space for data partitioning and mapping at each DRAM hierarchy level (i.e., from to each bank to channel), leading to a larger

TABLE IV
WORKLOADS FOR EVALUATIONS ((M, K, N) MEANS OPERAND SIZES ARE (M, K) AND (K, N), AND (M, N) IS THE MATRIX SIZE USED IN MVM.)

Model	Type	Size
AlexNet	MVM	(192, 1600), (256, 3456)
ResNet-18	MVM	(256, 2304), (512, 4608)
VGG-16	MVM	(256, 2304), (512, 4608)
Llama2-7B	MM	(4096, 4096, 4096), (11008, 4096, 4096), (4096, 11008, 4096), (4096, 128, 4096)
	MVM	(4096, 4096), (11008, 4096), (4096, 11008)
Llama2-13B	MM	(5120, 5120, 4096), (13824, 5120, 4096), (5120, 13824, 4096), (4096, 128, 4096)
	MVM	(5120, 5120), (13824, 5120), (5120, 13824)
Llama2-34B	MM	(6656, 6656, 4096), (832, 6656, 4096), (20480, 6656, 4096), (6656, 20480, 4096)
	MVM	(6656, 6656), (832, 6656), (20480, 6656), (6656, 20480)

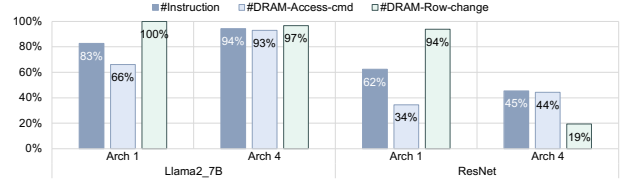


Fig. 8. Key metrics of compilation results (normalized to baseline).

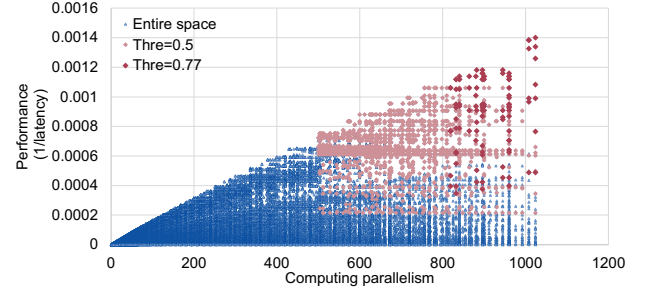


Fig. 9. Example of search space pruning.

optimization potential. This allows UniNDP to better leverage its advantages in partitioning and mapping multi-dimensional data onto multi-hierarchy level DRAM. Similarly, due to the specific circuit design of **Arch 2** and **Arch 3**, the compilation optimization for these architectures mainly happens at the device-level. Besides, the circuit design constraints the way to share data stored in the global buffer. Therefore, the compilation optimization space for these architectures is relatively limited. Existing compilation strategies are already quite effective, and the improvement provided by UniNDP is relatively modest. We also construct a channel-level NDP architecture based on the parameters of **Arch 5**. Compared to the baseline compilation strategy, UniNDP achieves only a 0.1–2% performance improvement. Unlike bank-level NDP which can be optimized for each bank individually, channel-level NDP treats multiple ranks as a whole part and behaves like traditional von Neumann architectures, leaving the minimum optimization space.

From the workload perspective, UniNDP achieves the

TABLE V
MM PERFORMANCE COMPARISONS BETWEEN BASELINE AND UNINDP (UNIT: NUMBER OF CLOCK CYCLE t_{CK})

	Arch1			Arch2			Arch3			Arch4			Arch5		
	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup
llama2_7B_MM1	6.98E+07	4.31E+07	1.62	4.55E+07	3.71E+07	1.23	8.41E+07	7.41E+07	1.14	6.15E+07	5.99E+07	1.03	4.55E+08	4.05E+08	1.12
llama2_7B_MM2	1.49E+08	1.15E+08	1.29	1.11E+08	9.95E+07	1.11	2.15E+08	1.99E+08	1.08	1.66E+08	1.62E+08	1.03	1.14E+09	1.09E+09	1.05
llama2_7B_MM3	1.87E+08	1.15E+08	1.62	1.23E+08	1.01E+08	1.22	2.27E+08	2.01E+08	1.13	1.65E+08	1.61E+08	1.03	1.22E+09	1.09E+09	1.12
llama2_13B_MM1	8.71E+07	6.71E+07	1.30	6.86E+07	5.84E+07	1.17	1.28E+08	1.17E+08	1.10	9.65E+07	9.42E+07	1.02	6.95E+08	6.33E+08	1.10
llama2_13B_MM2	2.19E+08	1.81E+08	1.21	1.73E+08	1.57E+08	1.10	3.27E+08	3.15E+08	1.04	2.53E+08	2.54E+08	1.00	1.77E+09	1.71E+09	1.04
llama2_13B_MM3	2.35E+08	1.81E+08	1.30	1.86E+08	1.62E+08	1.15	3.49E+08	3.16E+08	1.10	2.60E+08	2.54E+08	1.02	1.88E+09	1.71E+09	1.10
llama2_34B_MM1	1.56E+08	1.13E+08	1.38	1.20E+08	1.02E+08	1.18	2.14E+08	1.99E+08	1.08	1.62E+08	1.59E+08	1.02	1.15E+09	1.07E+09	1.08
llama2_34B_MM2	3.59E+07	1.44E+07	2.49	2.21E+07	1.26E+07	1.75	4.63E+07	2.50E+07	1.85	2.91E+07	1.99E+07	1.46	2.46E+08	1.34E+08	1.84
llama2_34B_MM3	3.70E+08	3.47E+08	1.07	3.19E+08	3.13E+08	1.02	6.27E+08	6.11E+08	1.03	4.94E+08	4.89E+08	1.01	3.37E+09	3.28E+09	1.02
llama2_34B_MM4	4.79E+08	3.48E+08	1.38	3.68E+08	3.13E+08	1.18	6.55E+08	6.06E+08	1.08	4.99E+08	4.90E+08	1.02	3.54E+09	3.29E+09	1.08
llama2_AttnHead	2.65E+06	1.55E+06	1.71	1.70E+06	1.45E+06	1.17	3.20E+06	2.90E+06	1.10	2.09E+06	1.94E+06	1.08	1.55E+07	1.40E+07	1.11
Average			1.49			1.21			1.16			1.07			1.15

TABLE VI
MVM PERFORMANCE COMPARISONS BETWEEN BASELINE AND UNINDP (UNIT: NUMBER OF CLOCK CYCLE t_{CK})

	Arch1			Arch2			Arch3			Arch4			Arch5		
	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup	Baseline	UniNDP	Speedup
llama2_7B_MVM1	17522	11482	1.53	11102	9494	1.17	20542	18640	1.10	15026	14674	1.02	111128	102728	1.08
llama2_7B_MVM2	36842	29549	1.25	27020	24722	1.09	52378	49096	1.07	40594	39266	1.03	277821	271314	1.02
llama2_7B_MVM3	46150	30246	1.53	30006	25646	1.17	55412	51012	1.09	40298	39582	1.02	298556	271514	1.10
llama2_13B_MVM1	21758	17342	1.25	16738	15158	1.10	31376	30160	1.04	23570	22858	1.03	169784	159182	1.07
llama2_13B_MVM2	53926	45220	1.19	42334	40334	1.05	79724	79700	1.00	61882	61422	1.01	432196	423772	1.02
llama2_13B_MVM3	57764	45682	1.26	45460	39110	1.16	85134	77952	1.09	63554	61566	1.03	457852	424008	1.08
llama2_34B_MVM1	38551	29152	1.32	29382	24710	1.19	52316	49168	1.06	39622	38548	1.03	280604	266920	1.05
llama2_34B_MVM2	9143	4149	2.20	5402	3378	1.60	11308	6580	1.72	6342	5161	1.23	60120	34903	1.72
llama2_34B_MVM3	90794	89178	1.02	77846	75254	1.03	152972	150256	1.02	118466	118114	1.00	821792	811856	1.01
llama2_34B_MVM4	117418	87658	1.34	89956	75506	1.19	159848	150688	1.06	121810	118082	1.03	863388	812022	1.06
alexnet_3_w	661	397	1.66	330	274	1.20	1000	596	1.68	462	386	1.20	7302	2433	3.00
alexnet_8_w	2088	859	2.43	874	642	1.36	1992	1312	1.52	1188	904	1.31	15654	6249	2.51
resnet18_3_w	1445	635	2.28	586	442	1.33	1416	952	1.49	776	645	1.20	10470	4305	2.43
resnet18_4_w	4523	1916	2.36	2430	1530	1.59	5268	3080	1.71	2954	2228	1.33	27788	15458	1.80
Average			1.62			1.23			1.26			1.10			1.50

highest speedup for the workload Llama2_34B_MM2 and Llama2_34B_MVM2, which are the attention operators (i.e., Grouped-Query Attention) in the prefill stage and decode stage, respectively. The main reason is that, compared to existing strategies, UniNDP not only partitions the weight dimension but also fully considers the overhead that may arise when the activation matrix gets larger. UniNDP can also optimize the compilation in the N dimension and K dimension. In real-world applications, long text large models are becoming increasingly important, and the computation of attention in the long context scenarios accounts for a significantly higher proportion. UniNDP can effectively enhance the computation speed of attention, making it promising for accelerating LLMs.

C. Simulator Validation

To verify the accuracy of UniNDP’s simulator, we use PIMSImulator [51] from Samsung as a baseline and use the same DRAM timing parameters, architecture design, and compilation strategy for comparison. Across multiple matrix operators, UniNDP’s simulation results exhibit a 6–8% deviation compared to PIMSImulator. The reason is that the modeling of DRAM refresh mechanism needs further refinement. Importantly, this error does not compromise the order-preserving comparisons, so the compiler’s optimization can be ensured. UniNDP’s instruction-driven simulator can achieve $1.62\text{--}1.82\times$ simulation speedup compared with PIMSImulator using cycle-by-cycle analysis, from $30.89s$ to $18.28s$.

D. Experiments for Search Space Pruning

To validate the effectiveness of our search space pruning method, a case study is conducted on the entire search space of a MVM operator with the size of 1000 on **Arch 2** to demonstrate how the search space is gradually pruned. As shown in Fig. 9, the search space is pruned with the indicator in Equation 8 with different thresholds ($0.5\times$ and $0.77\times$ of the highest performance upper bound in the search space). The result indicates that a search space with a smaller size can be produced by raising the threshold for pruning, while preserving the design points with better performance. Aside from that, the result also demonstrate that the performance of NDP architectures are bounded by the computing parallelism, therefore, we also prune the space by allocating computing parallelism as much as possible. Regarding the compilation time, we use the MM operator in Llama2-7B on **Arch 2** and **Arch 3** as an example. The search space pruning strategy can reduce the average compilation time from $5,640s$ to $1,475s$, achieving $3.82\times$ search efficiency improvement. It should be noted that the search space pruning primarily optimizes the stage that selects compilation results with top- K predicted performance, which can be reduced from $4,200s$ to $35s$. The following evaluation time for these top- K candidates using simulator will not be affected and become the major time-consuming part for compilation.

E. Predictor Accuracy and Speedup

In order to validate the accuracy of the proposed indicator, we compare the evaluation results generated by the predictor

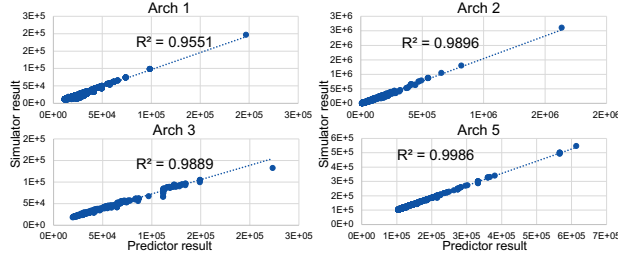


Fig. 10. Comparisons of evaluation results between predictor and cycle-accurate simulator for different NDP architectures.

and the cycle-accurate simulator. The comparison results are shown in Fig. 10. Here, we choose **Arch 1** (bank-level), **Arch 2 & Arch 3** (device-level), and **Arch 5** (rank-level) as examples. According to the results, the predictor’s predictions show a high correlation with the simulation results, with the coefficient of determination (R^2) values exceeding 0.95. Therefore, we can conclude that the predictor can provide performance predictions with a high degree of accuracy. Furthermore, the evaluation time for each compilation strategy of the predictor is $66\mu s$, achieving nearly $1.3 \times 10^6 \times$ evaluation speedup compared to the simulator. It should be noted that, although the predictor can quickly provide relative accurate evaluation results, it cannot rank the top- K results precisely. Only using predictor (i.e., w/o simulator) for compilation will cause 15% performance reduction. Therefore, the predictor is primarily used in the early design space exploration phase to select the top- K candidates. Subsequent performance comparison and selection for the best compilation strategy still require the cycle-accurate simulator to ensure the best compilation result. However, since only the top- K results need to be accurately simulated, the longer simulation time of the simulator remains acceptable.

F. Breakdown Study

Fig. 11 shows the time proportion for an MM operator on **Arch 1-3**, the operator size is $((M, K, N) = (832, 6656, 4096))$. Compared to the baseline strategy, UniNDP achieves the significant reduction in buffer read/write and the data communications between host and NDP. After the optimization, the most time-consuming part is the PU computing, and PU computing and DRAM access are executed in parallel

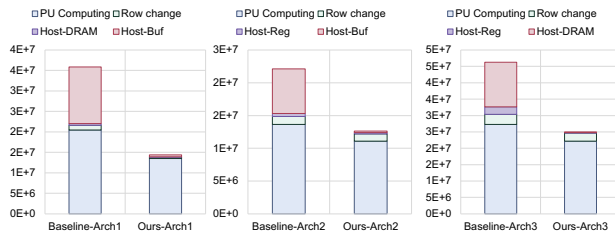


Fig. 11. Time proportion for MM on Arch 1-3.

using the out-of-order execution. Therefore, UniNDP achieves higher computing and internal bandwidth utilization.

G. NDP Design Insights

Insight 1: Sources of UniNDP’s performance gains and optimization directions for NDP compilation. UniNDP’s performance gains primarily come from two aspects: reducing the communication volume between the host and the NDP and the number of DRAM row activations. We observe that the former is more readily achieved by optimizing data partitioning and mapping and packing data/instructions. However, its gains exhibit diminishing marginal utility. When the communication between host and NDP can be completely overlapped by NDP computation, further performance improvements need to be attained by reducing the number of DRAM row activations.

Insight 2: Buffer design philosophy for NDP. Increasing the input buffer size of PUs can store more input data to enhance the DRAM data reuse, reducing the number of DRAM row activations. For example, when increasing the PU’s input buffer size from 32B to 512B for **Arch 4**, the performance of MM and MVM can be improved by $4.36\times$ and $4.55\times$, respectively. However, the output data is segmented for parallel processing, resulting in lower requirements for the output buffer size. Increasing the output buffer only yields a 0.65% performance improvement.

Insight 3: Comparisons of different NDPs. With the same total computational power and DRAM parameters, the absolute performance differences among different NDP categories are not significant. Device-level NDP and rank-level NDP exhibit slightly better performance than bank-level and channel-level NDP. When the total buffer size is the same, the individual buffer size for each PU in bank-level NDP is relatively small. While the buffer data of device-level NDP can be shared among multiple PUs, effectively increasing the equivalent buffer size. Thus, device-level NDP can achieve up to a 39% performance improvement over bank-level NDP.

Insight 4: Optimal data layout and compilation strategies. The data layout is closely related to the buffer size and whether multiple PUs share data from a single buffer. Before the input buffer is replaced, the required weight data should be arranged within the same DRAM row to minimize the number of DRAM row activations and maximize the input data reuse. When the input matrix is large, data should be partitioned along the input dimensions, rather than the reduction dimension, to avoid additional reductions for outputs.

VIII. RELATED WORK

With the model size of neural networks grow exponentially in recent years, the memory wall problem has become the main bottleneck for the hardware performance. Near-DRAM-Processing (NDP) architectures which place processing units in the main memory shows great potential to solve this problem by fully utilizing the internal bandwidth of DRAM [2], [3], [7], [10], [11], [15], [17], [18], [23], [26], [28], [29], [32], [35], [36], [38], [39], [49], [54], [60]–[62], [71], [73], [77]. Several industrial and commercial NDP accelerators

have also been designed [26], [28], [35], [36]. Aside from the architectures and products, researchers have also work on the compilation and simulation tools to construct the software stack for NDP. *As to the compilation tools*, CAIRO [21] supports instruction-level PIM offloading to Hybrid Memory Cube (HMC), “To PIM or Not” [14] optimize on how to offload computations to a bank-level NDP architecture. PIM-Flow provide compiler and runtime support for convolutional neural networks on PIM-enabled GPU memory [55]. Other compilers [1], [27] also provide support for NDP architectures. *As to the Simulation tools*, uPIMulator [26] is proposed as a simulation framework for UPMEM-PIM, which supports the compilation of UPMEM-PIM source codes and the subsequent cycle-level simulation. Ramulator-PIM [58] combines ZSim [52] and Ramulator [30] to simulate the NDP architectures. PIMSimulator [51] are designed by Samsung to provide cycle-accurate simulation for HBM-PIM [36]. PRIMO [24] designs a FPGA emulator to accelerate the simulation process of NDP architectures. Comparing to these existing tools which are aimed on certain NDP architectures, our UniNDP provide the best architectural compatibility. UniNDP also provide an in-depth analysis for the partition and mapping strategies.

IX. CONCLUSION

NDP architectures have provided a promising solution to overcome the “memory wall” problem in ML applications. In order to fully exploit the advantages of NDP architectures, the strategies of data partition, data mapping, and workload scheduling should be optimized meticulously. In this paper, we propose UniNDP, a unified NDP compilation and simulation tool to enable the workload mapping optimizations onto different NDP architectures. To achieve this goal, we propose a unified tree-based NDP hardware abstraction, a cycle-accurate and instruction-driven NDP simulator, and an efficient NDP compiler using search space pruning and performance predictor. Extensive experimental results show that, compared to existing mapping and compilation methods, UniNDP can achieve $1.05\text{--}3.43\times$ end-to-end speedup across multiple NDP architectures on CNNs and LLMs. In the future, we will improve the applicability of UniNDP to support other non-ML workloads and general computations.

ACKNOWLEDGEMENTS

This work was supported by the National Key R&D Program of China (2023YFB4502200), the National Natural Science Foundation of China (No. 62325405, U21B2031, 62204164), Tsinghua EE Xilinx AI Research Fund, Tsinghua-Meituan Joint Institute for Digital Life, and Beijing National Research Center for Information Science and Technology (No. BNR2024TD03001). This research was partially supported by ACCESS – AI Chip Center for Emerging Smart Systems, sponsored by InnoHK funding, Hong Kong SAR (HKSAR) and Research Grants Council of HKSAR (16213824).

REFERENCES

- [1] H. Ahmed, P. C. Santos, J. P. Lima, R. F. Moura, M. A. Alves, A. C. Beck, and L. Carro, “A compiler for automatic selection of suitable processing-in-memory instructions,” in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2019, pp. 564–569.
- [2] M. Alian, S. W. Min, H. Asgharimoghaddam, A. Dhar, D. K. Wang, T. Roewer, A. McPadden, O. O’Halloran, D. Chen, J. Xiong *et al.*, “Application-transparent near-memory processing architecture with memory channel network,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 802–814.
- [3] B. Asgari, R. Hadidi, J. Cao, S.-K. Lim, H. Kim *et al.*, “Fafnir: Accelerating sparse gathering by using efficient near-memory intelligent reduction,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 908–920.
- [4] S. Bianco, R. Cadene, L. Celona, and P. Napoletano, “Benchmark analysis of representative deep neural network architectures,” *IEEE Access*, vol. 6, p. 64270–64277, 2018. [Online]. Available: <http://dx.doi.org/10.1109/ACCESS.2018.2877890>
- [5] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, “The gem5 simulator,” *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.
- [6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [7] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, “Simplepim: A software framework for productive and efficient processing-in-memory,” in *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2023, pp. 99–111.
- [8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [9] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, “{TVM}: An automated {End-to-End} optimizing compiler for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594.
- [10] J. Cong, Z. Fang, M. Gill, F. Javadi, and G. Reinman, “Aim: accelerating computational genomics through scalable and noninvasive accelerator-interposed memory,” in *Proceedings of the International Symposium on Memory Systems*, 2017, pp. 3–14.
- [11] G. Dai, Z. Zhu, T. Fu, C. Wei, B. Wang, X. Li, Y. Xie, H. Yang, and Y. Wang, “Dimining: pruning-efficient and parallel graph mining on near-memory-computing,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 130–145.
- [12] G. Dai, Z. Zhu, T. Fu, C. Wei, B. Wang, X. Li, Y. Xie, H. Yang, and Y. Wang, “Dimining: pruning-efficient and parallel graph mining on near-memory-computing,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 130–145.
- [13] J. Deng, W. Dong, R. Socher *et al.*, “Imagenet: A large-scale hierarchical image database,” in *CVPR*, 2009.
- [14] A. Devic, S. B. Rai, A. Sivasubramaniam, A. Akel, S. Eilert, and J. Eno, “To pim or not for emerging general purpose processing in ddr memory systems,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 231–244.
- [15] A. Farmahini-Farahani, J. H. Ahn, K. Morrow, and N. S. Kim, “Nda: Near-dram acceleration architecture leveraging commodity dram devices and standard memory modules,” in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2015, pp. 283–295.
- [16] B. E. Forlin, P. C. Santos, A. E. Becker, M. A. Alves, and L. Carro, “Sim2pim: A complete simulation framework for processing-in-memory,” *Journal of Systems Architecture*, vol. 128, p. 102528, 2022.
- [17] M. Gao, G. Ayers, and C. Kozyrakis, “Practical near-data processing for in-memory analytics frameworks,” in *2015 International Conference on Parallel Architecture and Compilation (PACT)*. IEEE, 2015, pp. 113–124.
- [18] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, “Tetris: Scalable and efficient neural network acceleration with 3d memory,” in

- Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 751–764.
- [19] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, “Ai and memory wall,” *IEEE Micro*, 2024.
 - [20] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, “Benchmarking a new paradigm: An experimental analysis of a real processing-in-memory architecture,” *arXiv preprint arXiv:2105.03814*, 2021.
 - [21] R. Hadidi, L. Nai, H. Kim, and H. Kim, “Cairo: A compiler-assisted technique for enabling instruction-level offloading of processing-in-memory,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 4, pp. 1–25, 2017.
 - [22] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
 - [23] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, “Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, 2024, pp. 722–737.
 - [24] J. Heo, Y. Shin, S. Choi, S. Yune, J.-H. Kim, H. Sung, Y. Kwon, and J.-Y. Kim, “Primo: A full-stack processing-in-dram emulation framework for machine learning workloads,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
 - [25] M. Horeni, P. Taheri, P.-A. Tsai, A. Parashar, J. Emer, and S. Joshi, “Ruby: Improving hardware efficiency for tensor algebra accelerators through imperfect factorization,” in *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2022, pp. 254–266.
 - [26] B. Hyun, T. Kim, D. Lee, and M. Rhu, “Pathfinding future pim architectures by demystifying a commercial pim technology,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 263–279.
 - [27] M. T. Kandemir, J. Ryoo, X. Tang, and M. Karakoy, “Compiler support for near data computing,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 90–104.
 - [28] S.-h. Kang, B. Kim, S. Lee, and K. Sohn, “An architecture of sparse length sum accelerator in axdim,” in *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2022, pp. 1–4.
 - [29] L. Ke, U. Gupta, B. Y. Cho, D. Brooks, V. Chandra, U. Diril, A. Firoozshahian, K. Hazelwood, B. Jia, H.-H. S. Lee *et al.*, “Recnmp: Accelerating personalized recommendation with near-memory processing,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 790–803.
 - [30] Y. Kim, W. Yang, and O. Mutlu, “Ramulator: A fast and extensible dram simulator,” *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
 - [31] Y. Kwon, G. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun *et al.*, “Memory-centric computing with sk hynix’s domain-specific memory,” in *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, 2023, pp. 1–26.
 - [32] Y. Kwon, Y. Lee, and M. Rhu, “Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 740–753.
 - [33] C. Lattner and V. Adve, “Llvm: A compilation framework for lifelong program analysis & transformation,” in *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 2004, pp. 75–86.
 - [34] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “Mlir: Scaling compiler infrastructure for domain specific computation,” in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 2–14.
 - [35] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim *et al.*, “A lynn 1.25 v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
 - [36] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin *et al.*, “Hardware architecture and software stack for pim based on commercial dram technology: Industrial product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 43–56.
 - [37] J. D. Leidel and Y. Chen, “Hmc-sim-2.0: A simulation platform for exploring custom memory cube operations,” in *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2016, pp. 621–630.
 - [38] C. Li, Z. Zhou, Y. Wang, F. Yang, T. Cao, M. Yang, Y. Liang, and G. Sun, “Pim-dl: Expanding the applicability of commodity dram-pims for deep learning via algorithm-system co-optimization,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, 2024, pp. 879–896.
 - [39] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, “Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, 2024, pp. 950–965.
 - [40] S. Li and B. Jacob, “Statistical dram modeling,” in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 521–530.
 - [41] S. Li, D. Reddy, and B. Jacob, “A performance & power comparison of modern high-speed dram architectures,” in *Proceedings of the International Symposium on Memory Systems*, 2018, pp. 341–353.
 - [42] S. Li, R. S. Verdejo, P. Radojković, and B. Jacob, “Rethinking cycle accurate dram simulation,” in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 184–191.
 - [43] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
 - [44] H. Luo, Y. C. Tu, F. N. Bostanci, A. Olgun, A. G. Ya, O. Mutlu *et al.*, “Ramulator 2.0: A modern, modular, and extensible dram simulator,” *IEEE Computer Architecture Letters*, 2023.
 - [45] O. Mutlu, “Memory-centric computing,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.20000>
 - [46] G. F. Oliveira, P. C. Santos, M. A. Alves, and L. Carro, “A generic processing in memory cycle accurate simulator under hybrid memory cube architecture,” in *2017 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*. IEEE, 2017, pp. 54–61.
 - [47] OpenAI, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
 - [48] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, “Timeloop: A systematic approach to dnn accelerator evaluation,” in *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2019, pp. 304–315.
 - [49] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “Attacc! unleashing the power of pim for batched transformer-based generative model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, 2024, pp. 103–119.
 - [50] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon *et al.*, “An lpddr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 970–982.
 - [51] Samsung Advanced Institute of Technology, “Pimsimulator,” <https://github.com/SAITPublic/PIMSimulator>.
 - [52] D. Sanchez and C. Kozyrakis, “Zsim: Fast and accurate microarchitectural simulation of thousand-core systems,” *ACM SIGARCH Computer architecture news*, vol. 41, no. 3, pp. 475–486, 2013.
 - [53] P. C. Santos, B. E. Forlin, and L. Carro, “Sim2pim: A fast method for simulating host independent & pim agnostic designs,” in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 226–231.
 - [54] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin *et al.*, “lanus: Integrated accelerator based on npu-pim unified memory system,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, 2024, pp. 545–560.
 - [55] Y. Shin, J. Park, S. Cho, and H. Sung, “Pimflow: Compiler and runtime support for cnn models on processing-in-memory dram,” in *Proceedings*

- of the 21st ACM/IEEE International Symposium on Code Generation and Optimization, 2023, pp. 249–262.
- [56] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
 - [57] G. Singh, L. Chelini, S. Corda, A. J. Awan, S. Stuijk, R. Jordans, H. Corporaal, and A.-J. Boonstra, “A review of near-memory computing architectures: Opportunities and challenges,” in *2018 21st Euromicro Conference on Digital System Design (DSD)*. IEEE, 2018, pp. 608–617.
 - [58] G. Singh, J. Gómez-Luna, G. Mariani, G. F. Oliveira, S. Corda, S. Stuijk, O. Mutlu, and H. Corporaal, “Napel: Near-memory computing application performance prediction via ensemble learning,” in *Proceedings of the 56th annual design automation conference 2019*, 2019, pp. 1–6.
 - [59] K. Sohn, W.-J. Yun, R. Oh, C.-S. Oh, S.-Y. Seo, M.-S. Park, D.-H. Shin, W.-C. Jung, S.-H. Shin, J.-M. Ryu *et al.*, “A 1.2 v 20 nm 307 gb/s hbm dram with at-speed wafer-level io test scheme and adaptive refresh considering temperature distribution,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 250–260, 2016.
 - [60] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, “Abc-dimm: Alleviating the bottleneck of communication in dimm-based near-memory processing with inter-dimm broadcast,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 237–250.
 - [61] B. Tian, Q. Chen, and M. Gao, “Abndp: Co-optimizing data access and load balance in near-data processing,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 3–17.
 - [62] B. Tian, Y. Li, L. Jiang, S. Cai, and M. Gao, “Ndpbridge: Enabling cross-bank coordination in near-dram-bank processing architectures,” 2024, to appear in Proceedings of the 51th Annual International Symposium on Computer Architecture.
 - [63] P. Tillet, H.-T. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19.
 - [64] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
 - [65] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
 - [66] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
 - [67] J. Vieira, N. Roma, G. Falcao, and P. Tomás, “gem5-ndp: Near-data processing architecture simulation from low level caches to dram,” in *2022 IEEE 34th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 2022, pp. 41–50.
 - [68] J. Vieira, N. Roma, G. Falcao, and P. Tomás, “Ndpmlator: Enabling full-system simulation for near-data accelerators from caches to dram,” *IEEE Access*, 2024.
 - [69] C. Wolters, X. Yang, U. Schlichtmann, and T. Suzumura, “Memory is all you need: An overview of compute-in-memory architectures for accelerating large language model inference,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.08413>
 - [70] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, “Sparseloop: An analytical approach to sparse tensor accelerator modeling,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1377–1395.
 - [71] Y. Wu, Z. Wang, and W. D. Lu, “Pim gpt a hybrid process in memory accelerator for autoregressive transformers,” *npj Unconventional Computing*, vol. 1, no. 1, p. 4, 2024.
 - [72] X. Xie, P. Gu, J. Huang, Y. Ding, and Y. Xie, “Mpu-sim: A simulator for in-dram near-bank processing architectures,” *IEEE Computer Architecture Letters*, vol. 21, no. 1, pp. 1–4, 2021.
 - [73] X. Xie, Z. Liang, P. Gu, A. Basak, L. Deng, L. Liang, X. Hu, and Y. Xie, “Spacea: Sparse matrix vector multiplication on processing-in-memory accelerator,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 570–583.
 - [74] Xilinx, “Ise simulator (isim),” 2009, <https://www.xilinx.com/products/design-tools/isim.html>.
 - [75] S. Xu, X. Chen, Y. Wang, Y. Han, X. Qian, and X. Li, “Pimsim: A flexible and detailed processing-in-memory simulator,” *IEEE Computer Architecture Letters*, vol. 18, no. 1, pp. 6–9, 2018.
 - [76] C. Yu, S. Liu, and S. Khan, “Multipim: A detailed and configurable multi-stack processing-in-memory simulator,” *IEEE Computer Architecture Letters*, vol. 20, no. 1, pp. 54–57, 2021.
 - [77] M. Zhou, W. Xu, J. Kang, and T. Rosing, “Transpim: A memory-based acceleration via software-hardware co-design for transformer,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1071–1085.

A. Abstract

This appendix provide guidance on reproducing the key results of UniNDP. Our artifacts include a github repo and example excel files demonstrating how to draw some of the figures. Samsung’s PIMSimulator [51] is utilized in the experiment workflow to validate the UniNDP simulator. Follow the steps in Appendix IX-D to set up these two code repositories.

We have provided a detailed workflow to get the results and figures in Sec.VII in Appendix IX-E. For the pruning and breakdown experiments, we also provide example excel files where data are already processed into figure. Besides, refer to Appendix IX-G for detailed usage of UniNDP.

B. Artifact check-list (meta-information)

- **Algorithm:** Multi-level Exhaustive Search, Cycle-accurate and Instruction-driven Simulation
- **Program:** Python, Shell, C++ (for simulator validation)
- **Compilation:** Scons
- **Run-time environment:** Python3
- **Hardware:** Linux Server with Multi-core CPU
- **Output:** CSV and Text log files
- **Experiments:** All experiments and figures in Sec.VII.
- **How much disk space required (approximately)?:** ~100MB
- **How much time is needed to prepare workflow (approximately)?:** 5 minutes
- **How much time is needed to complete experiments (approximately)?:** 2-3 days
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License
- **Archived (provide DOI)?:** Will be archived for final version.

C. Description

1) *How to access:* The code is publicly available at <https://github.com/UniNDP-hpca25-ae/UniNDP>. The example excel files are publicly available at google drive.

2) *Software dependencies:* Python3 & Scons on Linux OS.

D. Installation

1) *UniNDP:* follow the bash commands to install.

```
1 # download the repo
2 git clone git@github.com:UniNDP-hpca25-ae/UniNDP.git
3 cd UniNDP
4 pip install -r requirements.txt
```

2) *Samsung’s PIMSimulator [51]:* follow the commands and the README file of PIMSimulator to build it. *Note: PIMSimulator is only used for simulator validation.*

```
1 # bash
2 git clone git@github.com:SAITPublic/PIMSimulator.git
```

E. Experiment workflow

1) *MM & MVM on Arch 1-5 (Table V&VI):* First, run the following bash commands under the UniNDP dir:

```
1 # command format
2 bash ./process_workload.sh mm.csv {Arch} {topK} op
3 # example: test mm & mvm on AiM architecture, select
  top 30 results to simulate
4 bash ./process_workload.sh mm.csv aim 30 op
```

The adjustable arguments are as bellow:

- **Arch:** Arch 1-5 corresponds to upmem, aim, aim8, hbm-pim, dimmining.
- **topK:** how many compilation results are selected to be simulated, default=30, suggest to set to 30.

After all the workloads are compiled, run the following command to export the results into csv. For each architecture Arch, the results will be stored in UniNDP/op/mm_arch.csv.

```
1 cp script/combine_op.sh op/ # under UniNDP dir
2 cd op
3 bash combine_op.sh
```

2) End-to-End Experiment for Arch 1,2,4,5 (Fig. 7 & 8):

First, run the bash commands under UniNDP dir:

```
1 bash ./process_workload.sh {workload_name} {Arch} {
  topK} # command format, don't change line
2 bash ./process_workload.sh resnet_18_224.csv aim 30
  e2e # example: test resnet on AiM architecture,
  select top 30 results to simulate
```

The adjustable arguments are as bellow:

- **workload_name:** The name of workload file you want to compile, choose from: {resnet_18_224.csv, vgg11_224.csv, llama2_7B_decode_tk32.csv, llama2_7B_prefill.csv, llama2_13B_decode_tk32.csv, llama2_13B_prefill.csv}.
- **Arch:** Arch 1, 2, 4, 5 corresponds to upmem, aim, hbm-pim, and dimmining, respectively.
- **topK:** set to 30 for CNN and LLM decoding, set to 15 for LLM prefilling.

After all the workloads are compiled, run the following command to summarize the results into UniNDP/e2e/combined_results.csv.

```
1 cp script/combine_e2e.sh e2e/ # under UniNDP dir
2 cd e2e
3 bash combine_e2e.sh
```

The key metrics of the compilation results (Fig. 8) can also be found in the summarized csv file. Here are their corresponding names in the csv files: #instructions = cmd, #DRAM-Access-cmd = pu_dram_access+host_dram_access, #DRAM-Row-change = row_change.

3) *Simulator Validation (Sec. VII-C):* We compare simulation results of different MVM operators between UniNDP and Samsung PIMSimulator [51] to validate our simulator. As the Samsung’s simulator can only support workload size which is a multiple of 4096, we choose the following input size and output size of MVM for test.

input	4096	8192	16384	4096	16384	8192
output	4096	8192	16384	16384	4096	16384

For each input and output size, we compare the result and simulation latency of UniNDP and Samsung’s Simulator.

For UniNDP simulation, run the following bash command, and results and the latency of simulation will be reported in `verify_result/log/[input_size, output_size].log`.

```
1 python sim_verify.py -S {input_size} {output_size}
```

For Samsung's PIMSimulator, go to the `PIMSimulator/src/tests/PIMBenchTestCases.cpp` file, and change the `gemv` test case defined at **line 23** into:

```
1 // PIMBenchTestCases.cpp
2 TEST_F(PIMBenchFixture, gemv)
3 {
4     setPIMBenchTestCase(KernelType::GEMV, {
5         output_size}, {input_size}); // change the
6         input_size and output_size here
7     executePIMKernel();
8 }
```

Then run the following commands under `PIMSimulator` dir to compile and simulate, results and the latency of simulation will be reported in the `PIMSimulator/log.txt`.

```
1 # compile the test
2 scons
3 # run the test
4 ./sim --gtest_filter=PIMBenchFixture.gemv > log.txt
```

4) Pruning Verification (Fig. 9): This figure is produced by compiling an MVM operator with the size of 1000 on Arch 2, and gradually restrict on the performance upper bound (0.5 $\times$ and 0.77 $\times$ of the highest performance upper bound in the search space).

```
1 python compile_detail.py -A aim -S 1 1000 1000 1 -O
   pruning_test -UU # [Under UniNDP dir]
```

Results are stored in `UniNDP/pruning_and_breakdown/pruning_test/csv/_gemm.csv`. As it's complicated to draw the figure, the processed excel file for this picture is provided in this link.

Moreover, the speedup of pruning can be tested through the following command. The detailed compilation time will be reported in log file.

```
1 # test the latency with pruning
2 python -OO compile_detail.py -A aim -S 4096 4096
   4096 1 -O with_prune -Q -K 30
3 # test the latency w/o pruning
4 python -OO compile_detail.py -A aim -S 4096 4096
   4096 1 -O no_prune -Q -K 30 -UU
```

5) Predictor Verification (Fig. 10): We verify the predictor accuracy for Arch 1, 2, 3, 5, on the MVM with the input size of 4096 and the output size of 4096 using the following command.

```
1 python compile_predictor.py -A {Arch} -S 1 4096 4096
   1 -O {Arch}_pred
```

The `{Arch}` can be chosen from `upmem`, `aim`, `aim8`, `dimming`. For each `{Arch}`, result csv will be generated in `./test_pred/{Arch}_pred/`. We can use predicted results and simulated results to verify the effectiveness of the predictor. We can also test the performance without the final

simulation stage, the speedup will decrease from 1.23 $\times$ to 1.18 $\times$, which is tested by the following command.

```
1 # test the performance with predictor and simulation
2 python -OO compile_predictor.py -A aim -S 4096 4096
   4096 1 -O with_sim -Q -K 30
3 # test the performance with only predictor, no
   simulation
4 python -OO compile_predictor.py -A aim -S 4096 4096
   4096 1 -O no_sim -Q -K 30 -NS
```

6) Breakdown (Fig. 11): The breakdown figure is draw from the command numbers in the result csv file (as well as the timing parameters in the config file). The processed excel file for this picture is provided in this link.

```
1 # {Arch} = upmem, aim, aim8
2 python -OO compile_detail.py -A {Arch} -S 4096 6656
   832 1 -O {Arch}_breakdown -Q -K 30
```

F. Evaluation and expected results

The results generated/plotted from this artifact should match those shown in result figures and tables. It should be noted that, as we improve the simulation accuracy during the rebuttal, the speedup results in Table V & VI will differ a little. Small differences might be observed in other experiments as well, but they do not influence the conclusion and insights of this paper. **The newest results will be updated in final paper.**

G. Experiment customization

1) Explore the Compiler: Use the `compile.py` to explore the compiler, and use `-h` option to see the detailed usage.

```
1 python compile.py -h # detailed usage
```

2) Explore the simulator: refer to `UniNDP/testsim.py`

3) Adding new architecture: directly modify the configuration file under `UniNDP/config` if you want to explore different architecture settings on existing computation dataflow. If not, you should copy `UniNDP/backend/_template.py` to add a new backend, which requires further efforts.

H. Notes

Some of the tests might take time (a few to ten hours) due to long simulation time of large operators (e.g., MMs in `llama2_34B`). Users can choose smaller `topK` than the suggested value to shorten search time, but the results might be sub-optimal.

Importantly, refer to this part of README file for how to monitor and terminate commands issued by the script.

I. Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>